Quantstamp

Parity Finance

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Stablecoin, Staking
Timeline	2024-08-19 through 2024-09-10
Language	Rust
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Specification	Parity
Source Code	• Parity-Finance/parity-contracts #9eee7b5 • Parity-Finance/parity-

Documentation quality	Medium
Test quality	Medium
Total Findings	8 Fixed: 7 Acknowledged: 1
High severity findings	0

	contracts #1acb182
Auditors	• Danny Aksenov Senior Auditing Engineer • Nikita Belenkov Auditing Engineer • Mustafa Hasan Senior Auditing Engineer

Medium severity findings	1 Fixed: 1
Low severity findings	4 Fixed: 4
Undetermined severe	1

rity findings	Fixed: 1
Informational findings	2 Fixed: 1 Acknowledged: 1

Summary of Findings

This audit report, prepared by Quantstamp, provides a security assessment of the Parity Finance stablecoin and staking programs. Overall, we found that the code is well-written, follows best practices, and is structured in a clear and maintainable way. Importantly, we did not identify any critical security vulnerabilities during the audit.

While the codebase is robust, there is room for improvement, particularly in the area of testing. We recommend increasing the depth and breadth of the testing framework to include edge cases that will ensure the system's reliability across a wider range of scenarios.

Please note that in this audit, QS has focused on the security of smart contracts and the governance processes of the protocol but has not explicitly assessed the particular investment strategies employed by Parity Finance.

Update:

The Parity Finance team has resolved all identified issues from the audit, fixing the medium and low-severity findings with improved validation and checks. Informational issues were either addressed or acknowledged as intentional design

choices. These updates have made improvements on the protocol's security and reliability. For further improvements, we recommend the team continue to improve their testing scope and coverage to ensure more robust and comprehensive resilience against further potential edge cases.

ID	DESCRIPTION	SEVERITY	STATUS
PAR-1	State-Account Balance Discrepancies in Token Management	Medium	Fixed
PAR-2	Missing Validation of Mint Address	Low	Fixed
PAR-3	Overwriting Pending Withdrawal Amount	Low	Fixed
PAR-4	Double Accounting in Current Slot Volume	Low	Fixed
PAR-5	Insufficient Input Validation Across Multiple Contract Functions	Low	Fixed
PAR-6	Missing Validation for `base_mint account`	Informational	Acknowledged
PAR-7	Potential Inconsistency in Deposit Cap Update	Informational	Fixed

ID	DESCRIPTION	SEVERITY	STATUS
PAR-8	**Inconsistent Admin Authority Checks in Token Management Functions**	• Undetermined	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

> **Disclaimer**
> Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Mishandled exceptions
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Seed collisions
- Arbitrary CPI
- Type cosplay
- Account reloading
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Missing account validation
- Appropriate ownership checks
- Arbitrary token minting
- Proper account initialization
- Proper account closing

Methodology

1. Code review that includes the following
    1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
    2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
    3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
    1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
    2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

During the audit, we reviewed the parity-issuance and parity-staking programs at commit `9eee7b5f08dfde09693467fa2facf55fe4c623c1` . At the time, work on pt-staking was ongoing and was subsequently reviewed at commit `1acb18214404cec647e3d91201ed932ee96f93aa` .

Files Included

Repo: `https://github.com/Parity-Finance/parity-contracts/tree/main/programs(9eee7b5f08dfde09693467fa2facf55fe4c623c1)`

Files:

- `programs/parity-issuance/*`
- `programs/parity-staking/*`

Repo: `https://github.com/Parity-Finance/parity-contracts/tree/main/programs(1acb18214404cec647e3d91201ed932ee96f93aa)`
Files:

- `programs/pt-staking/*`

Operational Considerations

1. Race conditions surrounding updating deposit cap via `update_pool_manager()` and `stake()`.
2. The hard-coded 1:1 exchange rate in the Issuance program could create arbitrage opportunities and potential abuse by the protocol's users if either PUSD or USDC depegs.
3. The Parity protocol interacts with centralized exchanges (CEXs), resulting in the fact that not all reserve management operations are fully visible on-chain. However, according to the provided documentation, Parity is committed to ensuring transparency in its CEX-based operations as well.
4. The minting functionality in the Parity Issuance program restricts minting and redemption of tokens to whitelisted users, based on a provided Merkle tree. PUSD, a permissionless token, is primarily acquired by users through secondary market transactions on decentralized exchanges (DEXs). While we appreciate Parity's implementation of the whitelist feature for enhancing security and compliance, we believe it also introduces some additional operational complexity for Parity.

Key Actors And Their Capabilities

Owner

- Can transfer ownership
- Can update admin
- Can update `Minter` (Stake Pool address which is allowed to mint PUSD without depositing USDC into the pool)
- Can update emergency fund basis point (% of USDC backing PUSD that always has to stay in the program for redemptions)
- Can update `withdrawTimelock` (time in seconds after which an initiated withdraw can be executed)
- Can update `WithdrawExecutionTime` (time in seconds after withdrawal becomes available it expires again)
- Update Metadata of stablecoin token
- Update metadata of `xMint`
- Controlled by a Fireblocks multi-sig wallet

Admin

- Can grant/revoke `Minter`, `Redeemer`, `Gatekeeper` roles
- Can set the `mintLimitPerSlot`.
- Can set the `redeemLimitPerSlot`.
- Can withdraw and deposit USDC into the Issuance Pool
- Can update the `yield_rate` of the staking contract
- Controlled by a Fireblocks multi-sig wallet

Gatekeepers

- Can disable mint/redeem functionality,
- Maximum 5 Wallets

Minter/Redeemer:

- Can invoke Mint and Redeem instruction from the issuance program
- Checked with a merkle-tree allowlist that can be updated by the Admin role
- No limit of potential wallets

Findings

PAR-1
State-Account Balance Discrepancies in Token Management

Medium Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `eef2a9b27809735afdc7c1b1992cf111f01d26ff` .
The client provided the following explanation:

```
Use vault ata balance for any checks
```

Update

The client added additional instructions to withdraw excess funds in the following commits: `f222fd42c7099f1cd159606e4dec53f98185a489` and `5e1cf25bf8606896a29d56af0d4a98821fb61d09` .

File(s) affected: `programs/parity-issuance/src/instructions/deposit_funds.rs` , `programs/parity-issuance/src/instructions/withdraw_funds.rs` , `programs/parity-staking/src/instructions/unstake.rs` , `programs/parity-issuance/src/instructions/initialize_withdraw_funds.rs`

Description: The `parity-issuance` and `parity-staking` programs effectively track certain token-related changes locally in state variables such as `total_collateral` and `base_balance` . While there is nothing inherently wrong with tracking these values for program logic, they should not be relied upon as the authoritative source of truth for the actual token balances in vault accounts.

Discrepancies may occur because users can transfer tokens directly to vaults, resulting in a mismatch between the tracked state and the actual vault balance. For

example:

1. `deposit_funds.rs` **and** `withdraw_funds.rs` : The `total_collateral` field is updated locally in the `token_manager` account, but this value may diverge from the actual vault balance if tokens are transferred directly to the vault.
2. `unstake.rs` : The `base_balance` field in the `pool_manager` account is updated via a local variable, but it may not accurately reflect the true balance of the vault.
3. `initialize_withdraw_funds.rs` : Withdrawal validations are performed using the stored `total_collateral` value instead of the actual balance in the token account.

While local state tracking is beneficial for efficiency and logic within the program, it is important to recognize that these values do not always reflect the real-time balance of vaults and should not be considered definitive for account balances.

Recommendation: Ensure there is a clear distinction between the local state variables used for tracking operations and the actual vault balances. The program logic should be explicit in its use of these values, and where critical operations like withdrawals are involved, consider validating against the real-time token account balance to prevent potential discrepancies

PAR-2
Missing Validation of Mint Address

Low Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `5df982ba4ce4744d743d0a62f02962a903a45111` .
> The client provided the following explanation:
>
> `Fixed`

File(s) affected: `programs/parity-issuance/src/instructions/withdraw_funds.rs`

Description: The `mint` account in the `WithdrawFunds` struct lacks proper validation of its address. This allows an admin to pass in any mint account with an inflated supply, potentially bypassing withdrawal limits.

Recommendation: Implement a check to ensure that the `mint` account's address matches the one associated with the `token_manager` .

PAR-3
Overwriting Pending Withdrawal Amount

Low Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `81d727551aa4a827bca8ed5f14f3dcdc76048577` , `541dc61daefc7903d5ee779beadc24e3e566dfaa` .
The client provided the following explanation:

```
Can only be overwritten once it's no longer valid
```

File(s) affected: `programs/parity-issuance/src/instructions/initialize_withdraw_funds.rs`

Description: If a withdrawal is initiated twice before the first one is processed, the second one would overwrite the first one's pending withdrawal amount.

Recommendation: Implement a check to ensure there's no existing pending withdrawal before initiating a new one. Alternatively, consider using a queue system for multiple pending withdrawals.

PAR-4
Double Accounting in Current Slot Volume

Low Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `6436fb946ceb197d4b5cb6f10e34d32983fc1134` .
The client provided the following explanation:

```
Double accounting resolved
```

File(s) affected: `programs/parity-issuance/src/instructions/mint.rs` , `programs/parity-issuance/src/instructions/redeem.rs`

Description: When a new slot is encountered, the `current_slot_volume` is updated twice: once in the `check_block_limit` function and again at the end of the handler function. This could lead to incorrect accounting of the current slot volume.

Recommendation: Review the logic for updating `current_slot_volume`. Ensure it's only updated once per transaction, preferably at the end of the handler function after all checks and operations have been completed successfully.

PAR-5
Insufficient Input Validation Across Multiple Contract Functions

Low Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `26fd1c8857c89f5df53f0211c95331af6672d1e4`, `eb8c7ba08595b975a606bfc913a1bb72b5c30bee`.
> The client provided the following explanation:
>
> ```
> Added several input validations as recommended
> ```

File(s) affected: `programs/pt-staking/src/instructions/pt_stake.rs`, `programs/pt-staking/src/instructions/initialize_global_config.rs`, `programs/pt-staking/src/instructions/update_global_config.rs`, `programs/parity-staking/src/instructions/update_pool_manager.rs`, `programs/parity-staking/src/instructions/initialize_pool_manager.rs`, `programs/parity-staking/src/instructions/unstake.rs`, `programs/pt-staking/src/instructions/pt_unstake.rs`, `programs/parity-issuance/src/instructions/initialize_token_manager.rs`, `programs/parity-issuance/src/instructions/mint.rs`, `programs/parity-issuance/src/instructions/update_token_manager_owner.rs`, `programs/parity-staking/src/instructions/update_annual_yield.rs`

Description: Several functions across the parity-contracts codebase lack comprehensive input validation for critical parameters. This includes missing checks for non-zero values, absence of bounds validation for numerical inputs such as fees, and lack of verification for account authorities and states. The affected areas span various operations such as staking, unstaking, minting, configuration updates, and token management.

Several explicit examples:

1. `parity_staking` doesn't check if the quantity is 0 in the following functions

    1. `stake()`
    2. `unstake()`
2. `pt_staking` doesn't check if the quantity is 0 in the following functions
    1. `pt_stake()`
    2. `pt_unstake()`
3. `parity_issuance` doesn't check if the quantity is 0 in the following functions:
    1. `deposit_funds()`
    2. `withdraw_funds()`
    3. `mint_admin()`
    4. `mint()`
    5. `redeem()`

Recommendation: Implement thorough input validation for all user-supplied parameters and critical values throughout the contract. This should include:

1. Ensuring non-zero values where appropriate (e.g., stake and unstake quantities)
2. Implementing bounds checks for numerical inputs (e.g., deposit caps, fee rates)
3. Validating account authorities and states before allowing sensitive operations
4. Adding checks for reasonable ranges and constraints specific to each parameter
5. Verifying the integrity and correctness of input data structures

PAR-6
Missing Validation for `base_mint account`

• Informational Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:
>
> ```
> Smart Contracts can be standalone
> ```

File(s) affected: `programs/pt-staking/src/instructions/initialize_global_config.rs`

Description: The `base_mint account` in the `InitializeGlobalConfig` struct is not validated. This could potentially allow the contract to be initialized with an incorrect or malicious mint account.

Recommendation: Add constraints to the `base_mint account` to ensure it is the correct token mint account.

PAR-7
Potential Inconsistency in Deposit Cap Update

Informational Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `878ad5617d4dcaef89c3a3cecef5e4751f6619b5` , `47dc0ad6073a640b34f40d94dab82231996cb9b1` .
The client provided the following explanation:

```
Deposit cap can't be set below currently deposited amount
```

File(s) affected: `programs/pt-staking/src/instructions/update_global_config.rs`

Description: The `new_deposit_cap` can be set to a value less than the existing deposit amount in the vault account, potentially leading to inconsistencies.

Recommendation: Implement a check to ensure that the new deposit cap is not less than the current total deposits in the vault.

PAR-8
Inconsistent Admin Authority Checks in Token Management Functions

Undetermined Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `e56ba9cdaf5e7be03f2678a2cda9bc2539100935` .
The client provided the following explanation:

```
Role management fixed as advised
```

File(s) affected: `programs/parity-issuance/src/instructions/add_gatekeeper.rs` , `programs/parity-issuance/src/instructions/remove_gatekeeper.rs` , `programs/parity-issuance/src/instructions/update_token_manager_admin.rs`

Description: Several token management functions in the parity-issuance program exhibit inconsistencies between the implemented admin checks and the documented behavior. Specifically, these functions are using `token_manager.owner` for admin verification, while the documentation states that `token_manager.admin` should be responsible for these operations. This discrepancy affects the following functions:

1. Adding gatekeepers
2. Removing gatekeepers
3. Updating token manager admin

Recommendation: To resolve this inconsistency, implement one of the following solutions:

1. Update the admin checks in all affected functions to use `token_manager.admin` instead of `token_manager.owner` . This aligns the code with the current documentation and ensures that the intended admin account has the proper authority.
2. If the current implementation using `token_manager.owner` is intentional, update the documentation to accurately reflect this behavior. Clearly state that the owner, not the admin, is responsible for these token management operations.

Auditor Suggestions

S1

Using `empty_merkle_root` as a Flag for No Restrictions Is Not a Good Practice

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `86cb8efa19d5049bb85af857fac4ed0f10901927` .
The client provided the following explanation:

```
Another bool flag added
```

Update

The client did add a bool flag in commit `86cb8efa19d5049bb85af857fac4ed0f10901927` . However there is currently no update functionality for this mechanism.

Update

> The client added the ability to update the boolean in commit `377e68e1a2459556085d19f1d8bfaf8f2d6cbefe`.

File(s) affected: `programs/parity-issuance/src/state.rs`

Description: Currently a Merkle root is used to indicate if a whitelist is present for the token minting. If the Merkle root is empty, hence `self.merkle_root == empty_merkle_root`, then there is no whitelist, and all addresses are allowed to mint. This could be easily confused with a whitelist being present and the Merkle root not being set yet.

Recommendation: Consider using a different flag to signify that whitelisting is in place.

S2 Code Improvements

Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `81d727551aa4a827bca8ed5f14f3dcdc76048577`.
> The client provided the following explanation:
>
> ```
> Was resolved within other fixes
> ```

File(s) affected: `programs/parity-issuance/src/instructions/update_token_manager_owner.rs`, `programs/parity-issuance/src/instructions/initialize_withdraw_funds.rs`

Description: The following improvements can be made to further the code quality:

1. In the `update_token_manager_owner` instruction, the line reading `token_manager.withdraw_execution_window = withdraw_execution_window` lacks a semi colon towards the end, which may result in a premature function return if the two values being set after this point should be set.
2. In the `initialize_withdraw_funds` instruction, the `quote_amount > token_manager.total_collateral` check is unnecessary since the amount is later on checked against the output of `calculate_max_withdrawable_amount()`.

Recommendation: Add the function implementation if this feature is required. Remove the function and the respective `remove_gatekeeper.rs` file in the instructions directory otherwise.

S3

Consider Having an Updatable Exchange Rate for Minting

Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `4e7cafad97890fa54136ab7ce6edf5310fef39b4` .
> The client provided the following explanation:
>
> ```
> Added functionality to change exchange rate later
> ```

File(s) affected: `programs/parity-issuance/src/instructions/update_token_manager_owner.rs`

Description: Currently, there is a set exchange rate between the quote token and the base token that is being minted. This exchange rate is set at creation and can not be updated.

Recommendation: Consider if it would be useful to be able to update this exchange rate without the need to redeploy the contract.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not post an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.

- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

- `1d1...49f ./error.rs`
- `ba0...e63 ./lib.rs`
- `c18...80f ./state.rs`
- `72e...227 ./init_pt_stake.rs`
- `bf9...002 ./initiate_update_global_config_owner.rs`
- `3fc...5ea ./update_global_config.rs`
- `ce8...4b3 ./mod.rs`
- `fd6...ca1 ./update_global_config_owner.rs`
- `5e0...10e ./pt_unstake.rs`
- `81f...463 ./pt_stake.rs`
- `505...027 ./initialize_global_config.rs`
- `e3d...969 ./error.rs`
- `c52...cbb ./lib.rs`
- `ad8...909 ./state.rs`
- `5d1...0e1 ./u64x64_math.rs`
- `e50...92a ./initialize_pool_manager.rs`
- `9ba...e9a ./update_annual_yield.rs`
- `159...93b ./stake.rs`
- `39b...679 ./update_xmint_metadata.rs`
- `4f3...b50 ./initiate_update_pool_owner.rs`
- `cb0...9ce ./mod.rs`

- 61c...74c ./update_pool_manager.rs
- 931...1dd ./unstake.rs
- e5e...857 ./update_pool_owner.rs
- 929...4a9 ./parity-issuance/src/error.rs
- cac...d98 ./parity-issuance/src/lib.rs
- c39...1fb ./parity-issuance/src/state.rs
- ce1...48b ./parity-issuance/src/instructions/update_token_manager_owner.rs
- e49...c67 ./parity-issuance/src/instructions/remove_gatekeeper.rs
- 9f3...d31 ./parity-issuance/src/instructions/add_gatekeeper.rs
- f48...a22 ./parity-issuance/src/instructions/toggle_active.rs
- 3fb...9da ./parity-issuance/src/instructions/initialize_withdraw_funds.rs
- e1f...163 ./parity-issuance/src/instructions/deposit_funds.rs
- 1bc...07c ./parity-issuance/src/instructions/redeem.rs
- 79f...a25 ./parity-issuance/src/instructions/update_mint_metadata.rs
- d7c...f29 ./parity-issuance/src/instructions/mint_admin.rs
- a11...994 ./parity-issuance/src/instructions/update_manager_owner.rs
- f64...8eb ./parity-issuance/src/instructions/mod.rs
- 0e3...557 ./parity-issuance/src/instructions/initialize_token_manager.rs
- 20e...d09 ./parity-issuance/src/instructions/withdraw_funds.rs
- 25a...106 ./parity-issuance/src/instructions/mint.rs
- bee...3da ./parity-issuance/src/instructions/initiate_update_manager_owner.rs
- a1b...150 ./parity-issuance/src/instructions/update_token_manager_admin.rs

Tests

- d48...7ee ./tests/parity-contract.ts
- 6a3...f3b ./tests/utils/utilts.ts

Toolset

The notes below outline the setup and steps performed in the process of this audit.

Setup

Tool Setup:

- Cargo Audit 0.20.1

Steps taken to run the tools:

- Installed via `cargo install cargo-audit`
- Ran `cargo audit`

Automated Analysis

Cargo Audit

Critical Vulnerabilities

1. curve25519-dalek (version 3.2.1)
    - **Title**: Timing variability in `curve25519-dalek`'s `Scalar29::sub` / `Scalar52::sub`
    - **ID**: RUSTSEC-2024-0344
    - **Solution**: Upgrade to >=4.1.3
1. ed25519-dalek (version 1.0.1)
    - **Title**: Double Public Key Signing Function Oracle Attack on `ed25519-dalek`
    - **ID**: RUSTSEC-2022-0093
    - **Solution**: Upgrade to >=2

Unsound Warnings

1. borsh (versions 0.9.3 and 0.10.3)
    - **Title**: Parsing borsh messages with ZST which are not-copy/clone is unsound
    - **ID**: RUSTSEC-2023-0033
1. ouroboros (version 0.15.6)
    - **Title**: Ouroboros is Unsound
    - **ID**: RUSTSEC-2023-0042

Unmaintained Packages

1. ansi_term (version 0.12.1)
    - **Title**: ansi_term is Unmaintained
    - **ID**: RUSTSEC-2021-0139
1. proc-macro-error (version 1.0.4)
    - **Title**: proc-macro-error is unmaintained
    - **ID**: RUSTSEC-2024-0370

Other Issues

1. atty (version 0.2.14)
   - **Title**: Potential unaligned read
   - **ID**: RUSTSEC-2021-0145

Summary

The cargo audit revealed two critical vulnerabilities, two packages with unsound warnings, two unmaintained packages, and one package with a potential unaligned read issue. It is strongly recommended to address these issues, particularly the critical vulnerabilities, by upgrading the affected packages to their suggested versions or finding alternative solutions.

Test Suite Results

```
  parity-contract
Created USDC:  4wiCJqtotDKdizx5hxkhEyk4cbMnKFRZ9HyveGbARqMp
    parity-issuance
      ✔ Token manager is initialized!
      ✔ pUSD can be minted for USDC (13441ms)
    Parity-staking
      Parity staking Tests
        ✔ Stake Pool is initialized!
        ✔ baseMint can be staked for xMint (13443ms)
        ✔ baseMint can be unstaked by redeeming xMint
(13429ms)
        ✔ should allow admin to withdraw excess tokens
(40320ms)
    Pt-staking
      Pt-staking Tests
        ✔ Global Config is initialized
        ✔ baseMint can be staked in PT Staking (26897ms)
        ✔ baseMint can be unstaked in PT Staking (26883ms)
        ✔ should allow admin to withdraw excess tokens in
PT Staking (40329ms)


  10 passing (4m)


 Running unittests src/lib.rs
(/home/ubuntu/workspaces/parity-
contracts/target/debug/deps/parity_issuance-
b57c4c81e6c17ed9)
```

```
running 5 tests
test state::tests::test_calculate_max_withdrawable_amount
... ok
test state::tests::test_calculate_normalized_quantity ...
ok
test state::tests::test_calculate_quote_amount ... ok
test state::tests::test_check_excessive_deposit ... ok
test test_id ... ok

test result: ok. 5 passed; 0 failed; 0 ignored; 0 measured;
0 filtered out; finished in 0.00s

Running unittests src/lib.rs
(/home/ubuntu/workspaces/parity-
contracts/target/debug/deps/parity_staking-
1c2e7ea68ddc20dd)

running 11 tests
test state::tests::test_calculate_amount_to_mint ... ok
test state::tests::test_calculate_exchange_rate ... ok
test state::tests::test_calculate_normalized_quantity ...
ok
test state::tests::test_calculate_output_amount ... ok
test state::tests::test_check_excessive_deposit ... ok
test test_id ... ok
test u64x64_math::tests::test_pow_large_base ... ok
test u64x64_math::tests::test_pow_overflow_exponent ... ok
test u64x64_math::tests::test_pow_positive_exponent ... ok
test u64x64_math::tests::test_pow_positive_exponent ... ok
test u64x64_math::tests::test_pow_zero_exponent ... ok

test result: ok. 11 passed; 0 failed; 0 ignored; 0
measured; 0 filtered out; finished in 0.00s

 Running unittests src/lib.rs
(/home/ubuntu/workspaces/parity-
contracts/target/debug/deps/pt_staking-bc58238573dc838f)

running 13 tests
test state::tests::test_calculate_points ... ok
test
state::tests::test_calculate_points_with_multiple_changes
... ok
test state::tests::test_calculate_points_with_phases ... ok
test state::tests::test_calculate_points_with_yield_change
... ok
test state::tests::test_check_excessive_deposit ... ok
```

```
test state::tests::test_get_current_exchange_rate ... ok
test state::tests::test_global_config_update_global_points
... ok
test state::tests::test_initialize_global_config ... ok
test state::tests::test_initialize_user_stake ... ok
test state::tests::test_update_global_config ... ok
test state::tests::test_update_user_stake ... ok
test state::tests::test_user_stake_update_points_history
... ok
test test_id ... ok

test result: ok. 13 passed; 0 failed; 0 ignored; 0
measured; 0 filtered out; finished in 0.00s
```

Changelog

- 2024-09-11 - Initial report
- 2024-09-27 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 500 audits and secured over $200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Near, Flow, Avalanche, Solana, Cardano, Binance Smart Chain, Hedera Hashgraph, Tezos
- DeFi: Curve, Compound, Maker, Lido, Polygon, Arbitrum, SushiSwap
- NFT: OpenSea, Parallel, Dapper Labs, Decentraland, Sandbox, Axie Infinity, Illuvium, NBA Top Shot, Zora
- Academic institutions: National University of Singapore, MIT

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites&aspo; owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF

DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and and may not be represented as such. No third party is entitled to rely on the report in any any way, including for the purpose of making any decisions to buy or sell a product, product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or or any open source or third-party software, code, libraries, materials, or information to, to, called by, referenced by or accessible through the report, its content, or any related related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.



© 2025 – Quantstamp, Inc.